

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Вятский государственный университет»
(ВятГУ)

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Рамешк Асли Амир Мохаммадович

(Ф.И.О. обучающегося)

27.03.04 Управление в технических системах, 02 Информационные технологии в системах
управления

(направление подготовки (специальность), направленность (профиль))

Место прохождения практики ФГБОУ ВО «Вятский государственный

(наименование организации, структурного подразделения организации)

университет», кафедра систем автоматизации управления

(наименование организации, структурного подразделения организации)

Итоговая оценка:

Руководитель
практики от университета

(дата)

(подпись)

А. Г. Симонов

(Ф.И.О.)

Киров, 2026 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Вятский государственный университет»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра систем автоматизации управления

**ПРОГРАММИРОВАНИЕ НА ЯЗЫКЕ PYTHON.
РЕАЛИЗАЦИЯ КРИПТОГРАФИЧЕСКОГО ШИФРА
«ШТАКЕТНИК»**

Отчет по учебной практике

ТПЖА.270304.259 ПЗ

Выполнил обучающийся
гр. УТб-2302-02-20

_____ / А. М. Рамешк Асли / _____

Руководитель

_____ / А. Г. Симонов / _____

Нормоконтролер

_____ / А. Г. Симонов / _____
(подпись) (инициалы, фамилия) (дата)

Киров
2026

Реферат

Рамешк Асли А.М. Программирование на языке Python. Реализация криптографического шифра «Штакетник». ТПЖА.270304.259 ПЗ: Учебная практика / ВятГУ, каф. САУ; рук. А. Г. Симонов – Киров, 2026. – ПЗ 59 с., 10 рис., 4 табл., 12 источников, 8 прил.

ЯЗЫК ПРОГРАММИРОВАНИЯ PYTHON, ИНТЕРПРЕТАТОР, БИБЛИОТЕКА NUMPY, МАШИННОЕ ОБУЧЕНИЕ, КЛАСТЕРИЗАЦИЯ, АЛГОРИТМ К-СРЕДНИХ, КЛАССИФИКАЦИЯ, АЛГОРИТМ К БЛИЖАЙШИХ СОСЕДЕЙ, ЕВКЛИДОВО РАССТОЯНИЕ, Z-НОРМАЛИЗАЦИЯ, КРИПТОГРАФИЯ, ШИФР ПЕРЕСТАНОВКИ, ШИФР «ШТАКЕТНИК», ШИФРОВАНИЕ, ДЕШИФРОВАНИЕ.

Объект исследования и разработки – алгоритмы обработки данных и криптографические алгоритмы, реализованные средствами языка программирования Python.

Цель работы – изучение базовых возможностей языка программирования Python и реализация алгоритмов кластеризации, классификации и криптографической защиты информации с практической проверкой их работоспособности.

Методы проведения работы – изучение технической литературы по тематике практики, освоение синтаксиса и стандартной библиотеки языка Python, разработка программного обеспечения с использованием библиотеки NumPy для научных вычислений, анализ и интерпретация полученных результатов.

В ходе выполнения работы изучены варианты установки интерпретатора Python, рассмотрены основные среды разработки и редакторы кода для языка Python, освоены базовые конструкции языка. Реализован алгоритм кластеризации k -средних, с помощью которого решена задача деления студентов на две учебные группы по их оценкам. Реализован алгоритм классификации k ближайших соседей, применённый к задаче определения вида обезьян по физическим параметрам. Разработана программа, реализующая шифр перестановки «Штакетник» с возможностями шифрования и дешифрования текста.

Результаты работы могут быть использованы при изучении основ программирования на языке Python, базовых алгоритмов машинного обучения и классических методов криптографии, а также в дальнейшей учебной и научно-исследовательской деятельности.

№ строки	Формат	Обозначение	Наименование	Количество листов	№ экз.	Примеч.
1			<u>Документация общая</u>			
2			Вновь разработанная			
3						
4	A4	ТПЖА.270304.259 ПЗ	Пояснительная записка	59		
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						
23						
24						
25						
26						
27						
28						
			ТПЖА.270304.259 ДКП			
Изм	Лист	№ докум.	Подпись	Дата	Программирование на языке Python. Реализация криптографического шифра «Штакетник» Литер Лист Листов 1 59 Кафедра САУ Группа УТб-2302-02-20	
Разраб.	Рамешк Асли					
Пров.	Симонов					
Н. контр.	Симонов					
Утв.						

Содержание

Введение.....	4
1 Введение в программирование на Python.....	6
1.1 Обзор технической литературы.....	6
1.2 Варианты установки интерпретатора Python	9
1.3 Среды разработки и редакторы кода для Python	10
1.4 Основы языка Python	12
1.5 Вывод по разделу 1	20
2 Решение задачи кластеризации.....	22
2.1 Задача кластеризации	22
2.2 Постановка задачи.....	22
2.3 Описание алгоритма k -средних	23
2.4 Решение задачи.....	26
2.5 Вывод по разделу 2	28
3 Решение задачи классификации	29
3.1 Задача классификации	29
3.2 Постановка задачи.....	29
3.3 Описание алгоритма k ближайших соседей.....	30
3.4 Решение задачи классификации	32
3.5 Реализация функции классификации	32
3.6 Результаты классификации	33
3.7 Вывод по разделу 3	34
4 Реализация криптографического алгоритма «Штакетник»	36
4.1 История возникновения и краткое описание шифра.....	36
4.2 Принцип работы алгоритма	37

					<i>ТПЖА.270304.259 ПЗ</i>			
<i>Изм</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>				
<i>Разраб.</i>		<i>Рамешк Асли</i>			Программирование на языке Python. Реализация криптографического шифра «Штакетник»	<i>Литер</i>	<i>Лист</i>	<i>Листов</i>
<i>Пров.</i>		<i>Симонов</i>					2	59
<i>Н. контр.</i>		<i>Симонов</i>				Кафедра САУ Группа УТб-2302-02-20		
<i>Утв.</i>								

4.3 Реализация шифра «Штакетник» на языке Python	42
4.4 Результаты работы программы.....	43
4.5 Особенности алгоритма.....	45
4.6 Вывод по разделу 4	45
Заключение	47
Приложение А (обязательное). Основные типы данных в Python.....	49
Приложение Б (обязательное). Листинг программного кода реализации алгоритма k -средних	50
Приложение В (обязательное). Таблица с данными для решения задачи классификации.....	51
Приложение Г (обязательное). Листинги программного кода алгоритма k ближайших соседей.....	52
Приложение Д (обязательное). Листинг программного кода реализации алгоритма шифрования «Штакетник»	54
Приложение Е (справочное). Перечень определений и терминов.....	56
Приложение Ж (справочное). Перечень сокращений	57
Приложение И (справочное). Библиографический список.....	58

Введение

Язык программирования Python – это один самых популярных языков общего назначения. Благодаря минималистичному синтаксису, высокому уровню читаемости кода и обширной экосистеме сторонних библиотек он применяется в широком спектре задач: от автоматизации рутинных операций и веб-разработки до научных вычислений, анализа данных, машинного обучения и информационной безопасности.

Актуальность настоящей работы обусловлена возрастающей ролью алгоритмов машинного обучения и методов защиты информации в современных системах управления. Алгоритмы кластеризации и классификации лежат в основе систем поддержки принятия решений, обработки больших данных, диагностики и распознавания образов. Криптографические алгоритмы, в свою очередь, обеспечивают защиту информации при её хранении и передаче. Понимание принципов работы данных алгоритмов и навыки их практической реализации являются необходимой основой для дальнейшего изучения более сложных современных методов.

Объектом исследования и разработки в настоящей работе являются алгоритмы обработки данных и криптографические алгоритмы, реализованные средствами языка программирования Python.

Цель учебной практики – это изучение базовых возможностей языка программирования Python и реализация на его основе алгоритмов кластеризации, классификации и криптографической защиты информации.

Для достижения поставленной цели в работе решаются следующие задачи:

- провести обзор технической литературы по тематике работы;
- изучить варианты установки интерпретатора Python и рассмотреть основные среды разработки для языка;
- освоить базовые конструкции языка Python: типы данных, операции, условные конструкции, циклы, обработку исключений, функции и модули;

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		4

– изучить теоретические основы алгоритма кластеризации k -средних и реализовать его на языке Python с решением практической задачи разделения студентов на учебные группы;

– изучить теоретические основы алгоритма классификации k ближайших соседей и реализовать его на языке Python с решением задачи определения вида обезьян по физическим параметрам;

– изучить принципы работы классических шифров перестановки на примере шифра «Штакетник» и реализовать программу шифрования и дешифрования текста на его основе;

– провести тестирование разработанных программ на наборах исходных данных и проанализировать полученные результаты.

Пояснительная записка состоит из четырёх разделов, заключения и приложений. В первом разделе представлен обзор технической литературы по тематике работы, рассмотрены варианты установки интерпретатора Python, описаны основные среды разработки и редакторы кода для языка, а также изложена справка по базовым возможностям языка с примерами реализации. Во втором разделе рассматривается решение задачи кластеризации с использованием алгоритма k -средних: приводится теоретическое описание алгоритма, представлена его реализация и проанализированы результаты. Третий раздел посвящён решению задачи классификации методом k ближайших соседей. В четвёртом разделе описывается реализация криптографического алгоритма «Штакетник»: приводится история возникновения шифра, принципы его работы, программная реализация на языке Python без использования специализированных криптографических библиотек, а также результаты тестирования программы шифрования и дешифрования. В заключении сформулированы основные выводы по проделанной работе.

1 Введение в программирование на Python

Язык Python – это кроссплатформенный высокоуровневый интерпретируемый язык программирования, нацеленный на повышение продуктивности разработки и читаемости кода. Синтаксис Python минималистичен, и в то же время язык обладает богатой стандартной библиотекой с обширным набором функций [2].

1.1 Обзор технической литературы

Перед началом выполнения учебной практики был выполнен обзор технической литературы по тематике работы. Изучаемая область включает несколько крупных направлений: общие сведения о языке программирования Python и его базовых конструкциях, библиотеки для научных вычислений и анализа данных, классические алгоритмы машинного обучения (в частности, методы кластеризации и классификации), а также криптографические алгоритмы. Для каждого направления были подобраны как фундаментальные монографии, так и современные учебные пособия.

1.1.1 Литература по языку Python

Базовый источник информации по языку Python – это официальная документация [1], размещённая на сайте python.org. Документация содержит описание синтаксиса языка, стандартной библиотеки, руководство по началу работы (The Python Tutorial), а также подробную справку по всем встроенным типам данных, функциям и модулям. Документация регулярно обновляется и считается авторитетным источником при работе с актуальными версиями интерпретатора.

Один из самых известных учебников по языку Python – это книга Марка Лутца «Изучаем Python» [2] – ведущего специалиста в области преподавания этого языка. Книга выдержала пять изданий и переведена на множество языков. Она представляет собой подробное руководство по основам языка: типам данных

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		6

и операциям, синтаксическим конструкциям, функциям, модулям и пакетам, классам и обработке исключений. Изложение материала основано на учебных курсах, которые автор ведёт на протяжении многих лет, что обеспечивает высокий уровень практической пользы.

В качестве справочного материала использовалось руководство по основным возможностям языка от Real Python [3] – образовательного ресурса с обширной коллекцией статей и руководств, охватывающих как базовые понятия, так и продвинутые темы Python-разработки. Материалы ресурса отличаются практической направленностью и большим количеством примеров кода.

Для углублённого изучения отдельных конструкций языка использовался ресурс PEP (Python Enhancement Proposals) [4] – официальная коллекция предложений по развитию языка, в которой формализованы стандарты оформления кода (PEP 8), концепции типизации и многие другие аспекты Python.

1.1.2 Литература по библиотекам анализа данных

При работе с алгоритмами кластеризации и классификации использовались библиотеки NumPy и matplotlib. Основным источником по библиотеке NumPy – её официальная документация [5]. В ней описаны все функции работы с многомерными массивами, статистические методы и операции линейной алгебры. Для построения визуализаций аналогично использовалась документация matplotlib [6].

Чтобы глубже разобраться в практическом применении научных библиотек Python, использовалась книга Уэса Маккинни «Python и анализ данных» [7]. Маккинни – создатель библиотеки pandas, поэтому материал в книге изложен глубоко и точно. В третьем издании рассматриваются вопросы переформатирования, очистки и обработки данных через NumPy, pandas и Jupyter – то, что напрямую относится к задачам этой работы.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		7

1.1.3 Литература по алгоритмам машинного обучения

Для теоретического обоснования реализованных в работе алгоритмов кластеризации и классификации использовались несколько источников. Современный обзор алгоритмов машинного обучения с практическими примерами реализации на Python представлен в материалах библиотеки `scikit-learn` [8] – наиболее популярной библиотеке Python для машинного обучения. Документация библиотеки содержит как теоретическое описание алгоритмов k -средних и k ближайших соседей, так и руководства по их применению.

Глубокое теоретическое изложение алгоритмов машинного обучения, в том числе методов кластеризации и классификации, представлено в открытом курсе лекций К. В. Воронцова «Машинное обучение» [9], читаемом в Школе анализа данных Яндекса. Материалы курса включают подробное обоснование математических основ алгоритмов, что было использовано при подготовке теоретических разделов настоящего отчёта.

1.1.3 Литература по криптографическим алгоритмам

При выполнении раздела, посвящённого реализации криптографического шифра «Штакетник», использовались материалы по классическим шифрам перестановки. Теоретическое описание шифров простой перестановки, к семейству которых относится «Штакетник», представлено в статье «Основы криптоанализа шифра простой перестановки» [10], где рассматриваются принципы построения таких шифров и методы их криптоанализа.

Практические примеры реализации классических шифров на языке Python приведены в статье «Шифрование сообщений в Python. От простого к сложному» [11]. Хотя эта статья посвящена шифру Цезаря, представленные в ней подходы к работе со строками и реализации криптографических алгоритмов на Python были использованы при разработке программы шифрования. Описание принципов работы непосредственно шифра «Штакетник», его исторический контекст и

пример распределения символов по строкам взяты из энциклопедической статьи [12].

1.1.4 Выводы по обзору

Анализ литературы показал, что выбранная тематика учебной практики обеспечена обширным учебным и научным материалом. Для каждого из решаемых направлений – изучения основ Python, реализации алгоритмов машинного обучения и работы с криптографическими алгоритмами – существуют как фундаментальные источники, так и современные учебные пособия и электронные ресурсы. Используемые в работе алгоритмы подробно описаны в литературе, что позволяет провести их корректную реализацию и интерпретировать полученные результаты.

1.2 Варианты установки интерпретатора Python

Существует несколько способов установки интерпретатора языка Python.

1.2.1 Официальный сайт python.org

Наиболее распространенный и универсальный способ установки интерпретатора языка Python. ПО для скачивания доступно на официальном сайте по адресу python.org/downloads для разных платформ.

1.2.2 Менеджеры версий

Менеджеры версий позволяют устанавливать несколько версий интерпретатора параллельно и переключаться между ними по необходимости. Ниже перечислены популярные менеджеры версий:

- `pyenv` – самое популярное решение с широкой поддержкой от сообщества;
- `asdf` – универсальный менеджер версий, поддерживающий множество языков программирования, в том числе Python;
- `uv` – менеджер, поддерживающий управление зависимостями проекта.

1.2.3 Дистрибутивы для научных вычислений

Специализированные дистрибутивы, имеющие возможность установки бинарных пакетов и сложных зависимостей. Можно выделить несколько крупных проектов:

- Anaconda – дистрибутив, включающий предустановленные библиотеки: NumPy, pandas, Jupiter и другие;
 - Miniconda – минимальная версия anaconda с возможностью установки библиотек через пакетный менеджер conda;
 - Mamba и Micromamba – высокопроизводительные альтернативы conda.
- При выполнении текущей работы использовался дистрибутив Anaconda.

1.3 Среды разработки и редакторы кода для Python

На момент написания работы существует около десяти сред разработки для Python. В данном пункте будут перечислены только самые известные среды разработки и текстовые редакторы.

1.3.1 PyCharm

PyCharm – разработка компании JetBrains, специализированная IDE для Python. Распространяется в двух редакциях: Community – бесплатная версия с открытым исходным кодом, и Professional – платная версия с расширенным функционалом.

PyCharm обладает рядом возможностей:

- мощный отладчик;
- рефакторинг кода;
- интеграции с системами контроля версий;
- интеграции с различными СУБД.

PyCharm обладает обширным функционалом. Основным недостатком на этом фоне становится производительность.

1.3.2 Spyder

Spyder – IDE с открытым исходным кодом, ориентированная на научные вычисления и анализ данных. Распространяется в составе дистрибутива Anaconda.

Spyder обладает следующим рядом возможностей:

- интерактивная консоль IPython;
- инспектор переменных и стека вызовов;
- графический отладчик;
- профилировщик кода.

1.3.3 Visual Studio Code

Бесплатный кроссплатформенный редактор кода от Microsoft. Один из наиболее популярных инструментов разработки в наши дни. Для работы с Python устанавливается официальное расширение Python, обеспечивающее автодополнение, отладку, линтинг и форматирование кода. Дополнительные расширения позволяют работать с Jupyter Notebook, контейнерами, удалёнными системами.

Visual Studio Code обладает большим количеством преимуществ:

- высокая производительность;
- обширная библиотека расширений;
- активная поддержка от Microsoft и сообщества;
- бесплатная форма распространения.

1.3.4 Редакторы кода

В своем большинстве редакторы кода мало чем отличаются друг от друга. Как правило, отличается интерфейс и некоторые специфические функции. Поэтому преимущества и недостатки редакторов кода приведены не будут.

Самые популярные редакторы кода на момент написания работы:

- SublimeText – коммерческий редактор кода, известный высокой скоростью работы и минималистичным интерфейсом
- Vim и NeoVim – консольные редакторы кода с долгой историей развития. Neovim – это современная реализация Vim с расширенными возможностями.
- Cursor – редактор кода, основанный на Visual Studio Code, с глубокой интеграцией искусственного интеллекта. Предоставляет функции автодополнения на основе ИИ, чат с языковыми моделями для генерации и рефакторинга кода.

1.4 Основы языка Python

Ниже приведена справка по основным возможностям языка Python

1.4.1 Переменные и типы данных

В Python, как и в любом другом языке программирования, можно объявлять переменные определенных типов данных. Стоит отметить, что Python – это динамически типизируемый язык программирования, поэтому тип переменной может измениться при присваивании переменной значения другого типа, либо при явном преобразовании типа данных. Основные типы данных Python представлены в таблице А.1.

Пример объявления переменной, массива и словаря представлены в листинге 1.1.

Листинг 1.1 – Объявление переменной, массива и словаря

```
a = 10 #объявление переменной  
b = [1, 2, 3] #объявление массива  
c = {'name': 'lemon', 'color': 'yellow'} #объявление словаря
```

Как было сказано ранее, Python является динамически типизируемым языком программирования. Узнать текущий тип переменной можно с помощью функции type. Пример использования функции type приведен в листинге 1.2.

Листинг 1.2 Пример использования функции type

```
>>> a = 1
>>> type(a)
<class 'int'>
>>> a = [1, 2, 3]
>>> type(a)
<class 'list'>
>>> a = {'a': 1, 'b': 2, 'c': 3}
>>> type(a)
<class 'dict'>
>>> a = True
>>> type(a)
<class 'bool'>
```

Ещё одной интересной особенностью Python как динамического языка является то, что элементы массива или словаря могут хранить значения произвольных типов. Пример этого показан в листинге 1.3.

Листинг 1.3 – Пример использования разных типов в массиве и словаре

```
>>> player = {'rating': 180, 'scores': [35, 18, 41], 'active': True}
>>> player['scores']
[35, 18, 41]
```

Полезной возможностью языка Python является использование кортежей – групп переменных. Кортежи используются, когда нужно вернуть несколько значений из функции. Пример использования кортежей представлен в листинге 1.4.

Листинг 1.4 – Пример использования кортежей

```
def max_(arr):
    max_idx, max_val = 0, 0

    for i in range(len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]
            max_idx = i

    return max_val, max_idx
res = max_([2, 5, 4, 1])
print(res[0], res[1])
```

Приведённый выше код интересен по нескольким причинам. Во-первых, он показывает, что можно инициализировать несколько переменных сразу, хотя

такой подход усложняет восприятие кода. Во-вторых, из примера видно, что для обозначения границ блоков кода используются не скобки или ключевые слова, а отступы. Именно эта особенность делает программы на Python наглядными. Общепринято для отступов блоков кода использовать пробелы, а не табуляцию.

1.4.2 Операции

В Python реализованы все базовые арифметические операции: сложение, вычитание, умножение, деление, возведение в степень, остаток от деления, а также круглые скобки для управления приоритетом вычислений. При этом некоторые операции имеют особенности, присущие именно Python:

- оператор / выполняет вещественное деление, возвращая результат типа float;
- оператор // выполняет целочисленное деление с отбрасыванием дробной части;
- оператор ** используется для возведения в степень;
- оператор + для строк и массивов выполняет конкатенацию (объединение).

Использование операций в Python представлено в листинге 1.5.

Листинг 1.5 – Использование операций в Python

```
print(5 / 2)           #2.5 - вещественное деление
print(5 // 2)          #2   - целочисленное деление
print(5 ** 2)          #25  - возведение в степень
print(3 + 4)           #7   - сложение чисел
print('3' + '4')       #'34' - конкатенация строк
print([1, 2, 3] + [4, 5]) #[1, 2, 3, 4, 5] - конкатенация массивов
```

1.4.3 Условия

Условия в Python записываются с использованием конструкции if-elif-else. После ключевого слова if следует условие и двоеточие. Код условного блока пишется со следующей строки с отступом. Ключевые слова elif (для дополнительных проверок) и else (для альтернативной ветви) являются необязательными.

В условиях могут применяться операторы сравнения (==, !=, <, >, <=, >=) и логические операторы (and, or, not), а также характерная для Python запись цепочек сравнений. Пример использования условных операторов представлен в листинге 1.6.

Листинг 1.6 – Пример использования условных операторов

```
#полный блок if-elif-else
height = int(input('Введите рост: '))

if height < 160:
    size = 'S'
elif height < 170:
    size = 'M'
elif height < 180:
    size = 'L'
else:
    size = 'XL'

print(size)

#цепочка сравнений
if 165 < height < 175:
    print('Средний рост')

#логические операторы
sun = True
rain = False

if sun and not rain:
    print('Можно выйти на прогулку')

#проверки на равенство и неравенство с помощью логических операторов сравнения
if 2 * 2 == 4 and 2 * 2 != 5:
    print('Прописные истины')
```

1.4.4 Циклы

В Python реализованы два типа циклов: while и for. Цикл while повторяет выполнение блока кода, пока выполняется заданное условие. Цикл for предназначен для перебора элементов коллекций – списков, строк, кортежей, словарей и других итерируемых объектов.

Для задания числовых последовательностей в цикле for обычно используется функция range(). Она может принимать от одного до трёх

аргументов: одно значение задаёт верхнюю границу (начиная с 0 с шагом 1); два – начало и конец последовательности; три – начало, конец и шаг изменения.

Управлять выполнением циклов можно с помощью ключевых слов `break` (прерывание цикла) и `continue` (переход к следующей итерации). Особенностью Python является возможность использования блока `else` после цикла `for` – он выполняется в том случае, если цикл завершился штатно (без прерывания через `break`). Пример использования циклов приведен в листинге 1.7.

Листинг 1.7 – Пример использования циклов в Python

```
#цикл while с условием выхода
while input('q для выхода: ') != 'q':
    print('Выполнение программы')

#цикл for по элементам списка
for item in [1, 2, 3]:
    print(item)

#цикл for по символам строки
for letter in 'String':
    print(letter)

#цикл for с функцией range (нечётные числа от 1 до 9)
for i in range(1, 10, 2):
    print(i)

#краткая форма записи для списка степеней двойки
powers_of_two = [2 ** i for i in range(9)]
print(powers_of_two)          #[1, 2, 4, 8, 16, 32, 64, 128, 256]

#использование continue для пропуска итерации
for letter in 'пивет':
    if letter == 'p':
        continue
    print(letter, end='')      #выведет пивет

#использование блока else после цикла for
for letter in 'привет':
    if letter == 'e':
        break
else:
    print('Буква "е" не встретилась')
```

Однострочная форма записи цикла `for` (генератор списков) широко применяется для создания массивов на основе вычисляемых значений и считается одной из идиоматических конструкций языка Python.

1.4.5 Обработка исключений

В процессе выполнения программы могут возникать исключительные ситуации – деление на ноль, обращение к несуществующему файлу, переполнение и другие ошибки. Для их корректной обработки в Python предусмотрен механизм исключений, аналогичный реализациям в других языках программирования.

Базовая обработка исключений осуществляется с помощью конструкции `try-except`. Блок `try` содержит код, в котором может произойти исключение, а блок `except` – код обработки исключения. Дополнительно могут использоваться блоки `else` (выполняется, если исключения не возникло) и `finally` (выполняется в любом случае, независимо от наличия исключения, и обычно применяется для освобождения ресурсов).

В Python определено множество стандартных типов исключений: `ZeroDivisionError` (деление на ноль), `OverflowError` (переполнение при арифметической операции), `ImportError` (ошибка импорта модуля), `ValueError` (некорректное значение), `TypeError` (несоответствие типов) и другие. В идеале для каждого типа исключений рекомендуется реализовать отдельный обработчик. Пример использования обработки исключений с помощью конструкции `try-except-finally` представлены в листинге 1.8.

Листинг 1.8 – Пример использования конструкции `try-except-finally` для обработки ошибок

```
try:
    a = int(input('Первый множитель: '))
    b = int(input('Второй множитель: '))
    result = a * b
except ValueError:
    print('Ошибка: введены некорректные аргументы')
except Exception as e:
    print(f'Произошла ошибка: {e}')
```

```

else:
    print(f'Результат: {result}')
finally:
    print('Завершение программы')

```

1.4.6 Создание собственных функций

Объявление функции в Python начинается с ключевого слова `def`, за которым через пробел следует имя функции и список её параметров в круглых скобках. После закрывающей скобки ставится двоеточие, а тело функции записывается с отступом. Возвращаемое значение указывается после ключевого слова `return`; функция может возвращать одно или несколько значений (в последнем случае значения автоматически упаковываются в кортеж).

Параметры функций могут иметь значения по умолчанию, благодаря чему допустимо вызывать функцию без явного указания этих параметров.

В листинге 1.9. функция `factorial` также демонстрирует рекурсию – вызов функции из самой себя – и сокращённую запись условного выражения (`x if condition else y`), которая является аналогом тернарного оператора из языка C++.

Листинг 1.9 – Пример создания собственных функций в Python

```

#функция, возвращающая кортеж из двух значений
def minmax(a, b):
    return (a, b) if a < b else (b, a)
print(minmax(5, 3))          #(3, 5)

#функция со значением параметра по умолчанию
def factorial(n=5):
    return n * factorial(n - 1) if n > 1 else 1
print(factorial())           #120 (используется значение по умолчанию)
print(factorial(3))          #6

```

1.4.7 Импорт функций из других файлов

Для использования функций из других файлов или сторонних библиотек применяется директива `import`. Существует несколько вариантов её использования, каждый из которых имеет свои особенности.

При обычном импорте `import math` к функциям модуля обращаются через имя модуля и точку. Можно импортировать и конкретные функции через

from import ... – тогда их можно вызывать напрямую, без указания имени модуля. Есть ещё вариант с импортом всех имён через символ «*» (from math import *), но его не рекомендуют использовать из-за возможных конфликтов имён между модулями.

На практике чаще всего модулям задают короткие псевдонимы через ключевое слово as. Так делают при работе со всеми популярными библиотеками: общепринятые псевдонимы – np для NumPy, pd для pandas, plt для matplotlib.pyplot. Пример импорта функций из других файлов приведён в листинге 1.10.

Листинг 1.10 – Пример импорта функций из других файлов в Python

```
#способ 1: импорт всего модуля
import math
print(math.sqrt(4))                # 2.0

#способ 2: импорт конкретных функций
from math import sqrt, sin
print(sqrt(4))                    # 2.0
print(sin(0))                     # 0.0

#способ 3: импорт с псевдонимом
import numpy as np
import matplotlib.pyplot as plt
print(np.sqrt(4))                 # 2.0
```

1.4.8 Комментарии

Комментарии в Python бывают двух видов: однострочные и многострочные. Однострочные комментарии начинаются с символа # и продолжаются до конца строки. Многострочные комментарии заключаются в тройные кавычки (одинарные или двойные) и часто используются для документирования функций, классов и модулей – такие комментарии называют строками документации (docstrings).

Строка документации, размещённая в начале функции, доступна программно через атрибут __doc__ и используется средствами автоматической

генерации документации, а также отображается в подсказках IDE. Примеры комментариев в Python представлены в листинге 1.11.

Листинг 1.11 – Примеры комментариев в Python

```
#однострочный комментарий

#пример многострочного комментария
def factorial(n):
    ...
    Рекурсивная функция вычисления факториала числа n.

    Параметры:
        n – натуральное число
    Возвращает:
        Факториал числа n
    ...
    return 1 if n == 1 else n * factorial(n - 1)

#доступ к строке документации функции
print(factorial.__doc__)
```

1.5 Вывод по разделу 1

В ходе работы были рассмотрены основные аспекты, связанные с началом работы с языком программирования Python.

Были изучены различные варианты установки интерпретатора: с официального сайта python.org, с помощью менеджеров версий (pyenv, asdf, uv) и с использованием специализированных дистрибутивов (Anaconda, Miniconda). При выполнении работы был выбран дистрибутив Anaconda, включающий все необходимые библиотеки и инструменты для разработки.

Был выполнен обзор сред разработки и редакторов кода – PyCharm, Spyder, Visual Studio Code, а также Sublime Text, Vim/Neovim и Cursor. Для каждого инструмента были выделены ключевые особенности и область применения.

Также была изучена справка по базовым возможностям языка: переменные и типы данных, операции, условные конструкции, циклы, обработка исключений, создание функций, импорт модулей и оформление комментариев. Были рассмотрены особенности Python, отличающие его от других языков программирования: динамическая типизация, использование отступов для

					ТПЖА.270304.259 ПЗ	Лист
						20
Изм	Лист	№ докум.	Подпись	Дата		

обозначения блоков кода, возврат нескольких значений через кортежи и генераторы списков.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		21

2 Решение задачи кластеризации

В данном разделе рассматриваются теоретические основы и практическая реализация задачи автоматической группировки данных. Основное внимание уделено разработке и тестированию алгоритма кластеризации k -средних на примере распределения студентов по учебным группам.

2.1 Задача кластеризации

Кластеризация – задача группировки множества объектов на подмножества (кластеры) таким образом, чтобы объекты из одного кластера были более похожи друг на друга, чем на объекты из других кластеров по какому-либо критерию [9].

Существуют различные алгоритмы кластеризации, в том числе иерархические и неиерархические. Самым известным неиерархическим алгоритмом является k -средних [9].

2.2 Постановка задачи

Предположим, требуется сформировать 2 группы студентов (УТ-11 и УТ-12) для обучения на специальности У. Известны оценки абитуриентов за тесты по физике и математике. Оценки приведены в таблице 2.1.

Таблица 2.1 – Оценки абитуриентов по физике и математике

Номер студента	Физика	Математика
1	4	4
2	3	3
3	5	3
4	2	3
5	5	5
6	3	2
7	2	4
8	4	5
9	5	4
10	2	2

Требуется реализовать алгоритм k -средних, с помощью которого выделить 2 кластера, описывающих формируемые группы студентов.

2.3 Описание алгоритма k -средних

Алгоритм k -средних выполняется в несколько последовательных этапов:

- а) задаётся количество кластеров k , которые требуется обнаружить;
- б) центры кластеров инициализируются случайным образом в пределах диапазона значений признаков;
- в) каждый объект приписывается к ближайшему центру кластера (используется евклидово расстояние);
- г) на основании объектов, вошедших в каждый кластер, центры кластеров пересчитываются;
- д) шаги 3 и 4 повторяются до стабилизации центров кластеров – момента, когда на очередной итерации все объекты остаются в тех же кластерах, что и на предыдущей.

Помимо стабилизации центров, в качестве критерия остановки могут использоваться также ограничение на максимальное число итераций или достижение приемлемой величины ошибки.

Схема алгоритма k -средних представлена на рисунке 2.2.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		24

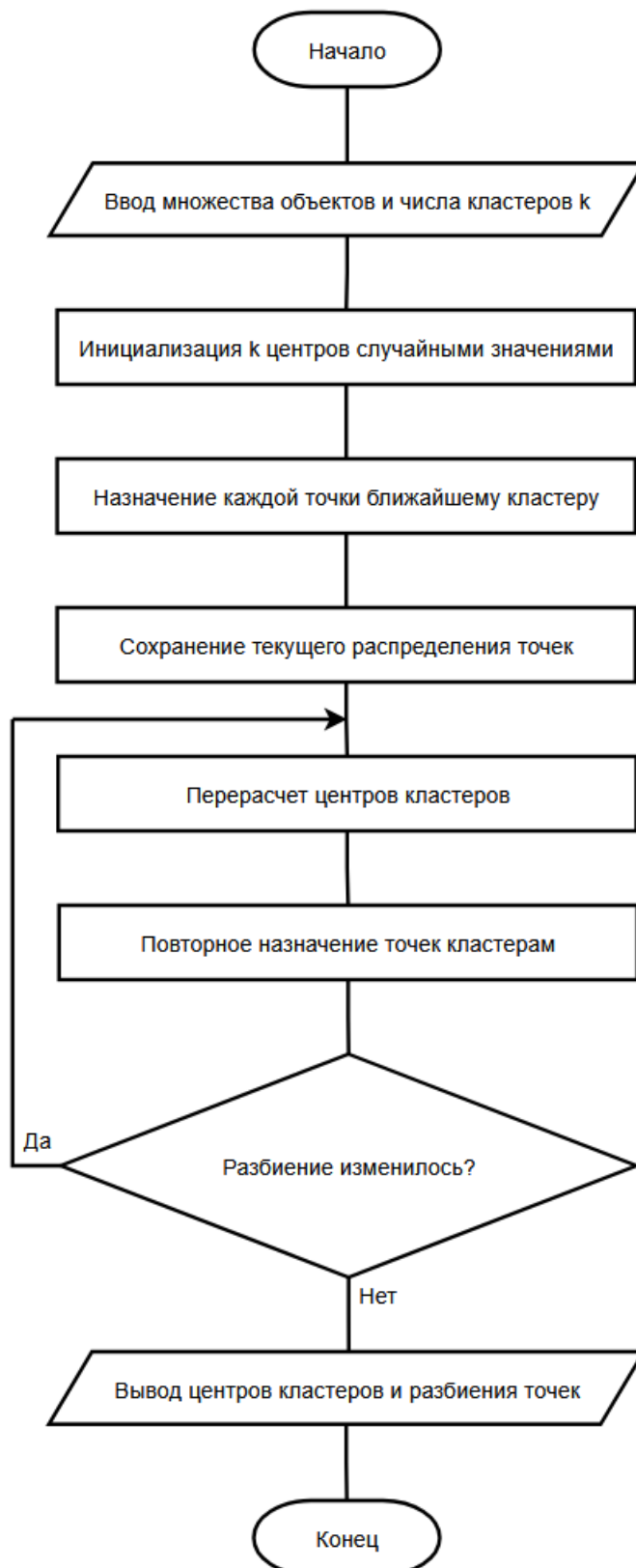


Рисунок 2.2 – Схема алгоритма k -средних

2.4 Решение задачи

Реализация алгоритма k -средних на языке Python с использованием библиотеки NumPy включает несколько этапов.

2.4.1 Вычисление евклидова расстояния

Расстояние между двумя точками A и B в N -мерном пространстве вычисляется по формуле [9]:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}. \quad (1)$$

Реализация функции на языке Python для расчета евклидова расстояния приведена в листинге 2.1.

Листинг 2.1 – Функция вычисления евклидова расстояния

```
def dist(A, B):  
    return np.sqrt(np.sum((A - B) ** 2))
```

2.4.2 Назначение точек кластерам

Для каждой точки вычисляется расстояние до всех центров кластеров и определяется индекс ближайшего центра. Эта операция реализована в функции `class_of_each_point`, использующей функцию `np.argmin` для поиска индекса минимального значения в каждой строке матрицы расстояний.

2.4.3 Итоговый код программы

Полный код программы представлен в листинге Б.1.

2.4.4 Результат работы алгоритма k -средних

После нескольких запусков программы было замечено, что при случайном выборе начальных центров кластеры могут незначительно отличаться, однако в

рассматриваемом наборе данных итоговое разделение оставалось практически одинаковым. Результаты работы программы представлена на рисунке 2.3.

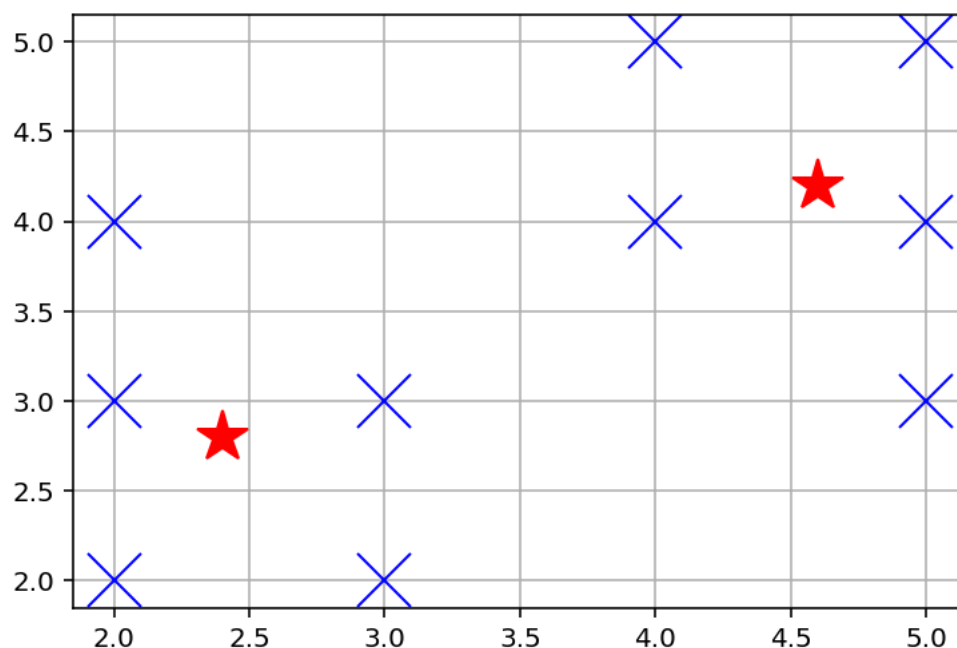


Рисунок 2.3 – Итоговое распределение кластеров

Первый кластер с центром в точке приблизительно (2.4; 2.8) объединяет студентов с более низкими оценками по обоим предметам. В него входят студенты с оценками (2; 2), (3; 2), (2; 3), (3; 3) и (2; 4) – то есть студенты № 10, 6, 4, 2 и 7 согласно таблице 2.1. Эта группа характеризует абитуриентов со средним уровнем подготовки.

Второй кластер с центром в точке приблизительно (4.6; 4.2) объединяет студентов с более высокими оценками. В него входят студенты с оценками (4; 4), (4; 5), (5; 3), (5; 4) и (5; 5) – студенты № 1, 8, 3, 9 и 5 согласно таблице 2.1. Эта группа характеризует абитуриентов с более высоким уровнем подготовки.

В каждый кластер вошло по пять студентов, что обеспечивает равномерное распределение учебной нагрузки между группами.

2.5 Вывод по разделу 2

В ходе работы был изучен и реализован один из наиболее популярных алгоритмов кластеризации – k -средних. Алгоритм был применён к практической задаче формирования двух учебных групп студентов на основе их оценок по физике и математике.

					ТПЖА.270304.259 ПЗ	Лист
						28
Изм	Лист	№ докум.	Подпись	Дата		

3 Решение задачи классификации

В данном разделе рассматриваются теоретические аспекты задачи обучения с учителем и её практическое решение на примере классификации биологических видов с помощью алгоритма k ближайших соседей.

3.1 Задача классификации

Классификация – это задача обучения с учителем. Её суть в том, чтобы отнести объект к одному из заранее известных классов на основании его признаков [8]. В отличие от кластеризации, рассмотренной в предыдущем разделе, при классификации метки классов для всех объектов обучающей выборки известны заранее. Алгоритм должен построить правило, по которому можно корректно определять класс новых, ранее не встречавшихся объектов.

Задачи классификации встречаются во многих областях: распознавание изображений и рукописного текста, медицинская диагностика, фильтрация спама в электронной почте, оценка кредитоспособности заёмщиков, определение тональности текстовых сообщений. Алгоритмов классификации много – среди них выделяют решающие деревья, наивный байесовский классификатор, метод опорных векторов, нейронные сети. Один из самых простых алгоритмов – метод k ближайших соседей, который и рассматривается в этом разделе.

3.2 Постановка задачи

В рамках данной работы решается следующая практическая задача. Профессор Буковски, продолжая изучение материалов из экспедиции в Африку, обнаружил результаты исследований японского учёного Какая Икота. Профессор Икота, изучая обезьян, выделил четыре класса: лемуры, шимпанзе, гориллы и орангутаны. Каждый вид характеризуется определёнными средними значениями роста и веса. Экспериментальные данные представлены в таблице В.1.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		29

Требуется реализовать алгоритм k ближайших соседей и с его помощью определить, к какому виду относятся обезьяны, изученные ассистентом профессора Буковски в ходе экспедиции.

3.3 Описание алгоритма k ближайших соседей

Перед началом классификации важно выполнить ещё один шаг – нормализацию признаков. В исходных данных признаки могут иметь сильно различающиеся диапазоны значений. В нашей задаче, например, рост принимает значения от 18 до 205, а вес – от 13 до 155. Без нормализации признаки с большими значениями будут оказывать непропорционально большое влияние на вычисляемое расстояние.

Чаще всего используют Z -нормализацию: из значений каждого признака вычитают среднее по столбцу и делят результат на среднеквадратическое отклонение. После такой обработки все признаки приводятся к одному масштабу – с нулевым средним и единичной дисперсией [8].

Схема алгоритма k ближайших соседей представлена на рисунке 3.1

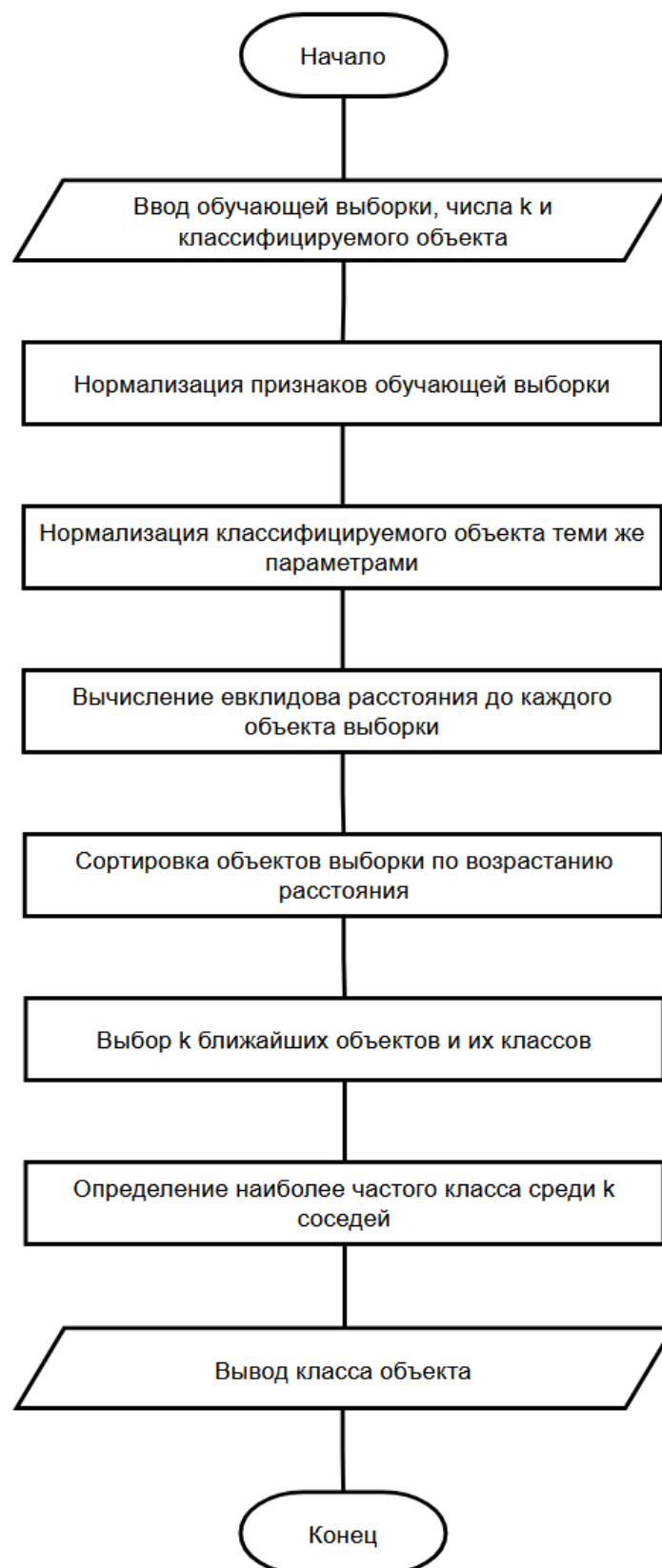


Рисунок 3.1 – Схема алгоритма k ближайших соседей

3.4 Решение задачи классификации

В файле `gun.py` была сформирована матрица X , содержащая обучающую выборку. Каждая строка матрицы соответствует одной особи: первый столбец содержит рост, второй – вес, третий – номер класса. Номер класса распределен от 1 до 4, согласно последовательности: лемур – шимпанзе – горилла – орангутанг). Также в файле организован ввод параметров особи с помощью функции `input` и преобразование введенных значений к целочисленному типу с помощью функции `int`. Полный код файла приведен в листинге Г.2.

3.5 Реализация функции классификации

В файле `kNN.py` была реализована функция `k_nearest`, выполняющая классификацию объекта методом k ближайших соседей. Функция последовательно выполняет следующие операции: выделение матрицы признаков из обучающего множества, нормализацию данных, нормализацию классифицируемого объекта с теми же параметрами, вычисление расстояний, сортировку соседей по удаленности, выбор классов k ближайших соседей и определение наиболее часто встречающегося класса. Полная реализация приведена в листинге Г.2.

В реализации функции `k_nearest` алгоритм работает следующим образом. Сначала из обучающей матрицы X выделяется матрица признаков – все столбцы, кроме последнего, в котором хранятся номера классов. Эта матрица признаков подвергается Z -нормализации. В итоге все признаки приводятся к единому масштабу.

Классифицируемый объект `obj` нормализуется с использованием тех же параметров среднего и среднеквадратического отклонения, что были рассчитаны для обучающей выборки.

Далее с помощью функции `dist` вычисляется евклидово расстояние от нормализованного объекта `obj` до каждой строки нормализованной матрицы

признаков. Получается массив расстояний, длина которого равна количеству объектов в обучающей выборке.

Полученный массив расстояний сортируется по возрастанию, при этом сохраняются индексы исходных строк. Это позволяет определить, какие именно объекты обучающей выборки оказались ближе всего к классифицируемому. Из отсортированного списка выбираются первые k индексов – это и есть k ближайших соседей.

По выбранным индексам из исходной (ненормализованной) матрицы X извлекаются метки классов – значения из последнего столбца. Среди этих меток определяется наиболее часто встречающаяся: подсчитывается количество вхождений каждого уникального класса, и выбирается тот, у которого это количество максимально. Полученный класс возвращается как результат классификации.

Результат работы программы с тестовыми данными (вес 100, рост 100) изображен на рисунке 3.1

```
In [3]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPractice/
_Материалы для прохождения учебной практики (ЗФО)/Методические указания УТ ЗФО 2026/
Методические указания УТ ЗФО 2026/3/run.py' --wdir
Reloaded modules: kNN
Введите рост особи: 100
Введите вес особи: 100
schimpanze
```

Рисунок 3.1 – Пример работы алгоритма

3.6 Результаты классификации

Для проверки работоспособности алгоритма была проведена классификация шести особей с различными значениями роста и веса. Использовалось значение параметра $k = 3$, заданное в файле run.py. Результаты классификации представлены в таблице 3.2.

Таблица 3.2 – Результаты классификации на тестовых данных

№	Рост	Вес	Результат классификации	Ожидаемый класс
1	30	20	Лемур	Лемур
2	75	125	Шимпанзе	Шимпанзе
3	190	140	Горилла	Горилла
4	150	50	Орангутан	Орангутан
5	45	18	Лемур	Лемур
6	100	100	Шимпанзе	Шимпанзе

Для всех шести тестовых наборов алгоритм корректно определил принадлежность к классу. Особи с малыми значениями роста и веса (30 см и 20 кг) были классифицированы как лемуры, что соответствует характеристикам этого вида. Особи с большим ростом и значительным весом (190 см, 140 кг) были классифицированы как гориллы. Особи с относительно высоким ростом, но малым весом (150 см, 50 кг) были корректно отнесены к орангутанам – виду, для которого характерно именно такое соотношение признаков. Особи со средним ростом и средним или большим весом классифицированы как шимпанзе.

3.7 Вывод по разделу 3

В ходе работы был изучен и реализован один из базовых алгоритмов классификации – метод k ближайших соседей. Алгоритм был применён к практической задаче определения вида обезьян по их росту и весу

Изучение алгоритма k ближайших соседей позволило освоить основные понятия задач классификации в машинном обучении: обучающая выборка, признаки объектов, нормализация данных, метрики расстояния. Также были

закреплены практические навыки работы с библиотекой NumPy: использование массивов, статистических функций, агрегирующих операций по осям матрицы и сортировки массивов с помощью функции argsort.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		35

4 Реализация криптографического алгоритма «Штакетник»

В данном разделе рассматриваются исторические аспекты, теоретические основы и программная реализация одного из классических методов симметричного шифрования. Основное внимание уделено алгоритмам перестановки символов и оценке их криптографической стойкости на примере шифра «Штакетник».

4.1 История возникновения и краткое описание шифра

Шифр «Штакетник» (англ. Rail Fence cipher, известен также как «зигзагообразный шифр» или «шифр железнодорожной ограды») относится к классическим шифрам перестановки. Это один из старейших и простых криптографических методов. Различные источники относят появление подобных шифров к временам Древней Греции – аналогичные методы перестановки символов использовались в эпоху античности для передачи военных и дипломатических сообщений.

Свое название шифр получил из-за визуального сходства с забором (штакетником) или железнодорожной оградой: при шифровании символы исходного текста размещаются на нескольких параллельных строках зигзагообразно – сверху вниз и обратно. Прочитав после этого символы построчно, получают зашифрованный текст.

Шифр относится к симметричным криптографическим алгоритмам: для шифрования и дешифрования используется один и тот же ключ. Ключом является целое число – количество строк, на которые распределяется текст. Из-за простоты и малого количества возможных ключей шифр считается крайне нестойким и в настоящее время не используется для защиты реальной информации, однако он представляет значительную образовательную ценность как пример классических шифров перестановки.

4.2 Принцип работы алгоритма

Функционирование криптографического алгоритма «Штакетник» базируется на циклическом изменении индексов строк при записи исходного сообщения. Процесс преобразования информации разделяется на два взаимнообратных этапа: формирование зашифрованного текста путём обхода условной матрицы и последующее восстановление исходных данных по известному ключу [12]. Ниже приведено детальное описание каждого из этих этапов.

4.2.1 Шифрование

Алгоритм шифрования выполняется в следующей последовательности [10], [12]:

- а) задаётся ключ k – число строк;
- б) создаётся k пустых строк;
- в) символы исходного текста последовательно распределяются по строкам зигзагом: первый символ – на верхнюю строку, затем движение направлено вниз до достижения последней строки, после чего направление меняется на противоположное (вверх до первой строки) и так далее.
- г) зашифрованный текст получается путём последовательного чтения всех строк сверху вниз и соединения их в одну строку.

Рассмотрим в качестве примера фразу «я очень люблю мороженое». После удаления пробелов получается строка «яоченьлюблюмороженое», состоящая из 20 символов.

В классическом варианте шифра с двумя строками полученный текст разбивается на две части равной длины. Если общее количество символов нечётное, к тексту добавляется произвольный символ-заполнитель. Символы распределяются по двум строкам зигзагом: первый символ – в верхнюю строку,

второй – в нижнюю, третий – снова в верхнюю и так далее. Результат распределения для рассматриваемого примера представлен на рисунке 4.1.

я		ч		н		л		б		ю		о		о		е		о	
	о		е		ь		ю		л		м		р		ж		н		е

Рисунок 4.1 – Результат распределения символов

Зашифрованный текст получается последовательным чтением строк: сначала верхняя строка, затем нижняя. Для рассматриваемого примера результат шифрования выглядит следующим образом:

ЯЧНЛБЮООЕО + ОЕБЮЛМРЖНЕ = ЯЧНЛБЮООЕООЕБЮЛМРЖНЕ

Полученный текст содержит те же буквы, что и исходное сообщение, однако их порядок изменён, что делает текст нечитаемым без знания алгоритма и ключа.

Дешифрование выполняется в обратном порядке. Получатель сообщения, зная ключ $k = 2$, делит зашифрованный текст на две равные части: *ЯЧНЛБЮООЕО* и *ОЕБЮЛМРЖНЕ*. Первая часть записывается в верхнюю строку, вторая – в нижнюю под ней. После этого исходный текст восстанавливается путём поочерёдного чтения символов из верхней и нижней строк: первый символ из верхней, первый из нижней, второй из верхней, второй из нижней и так далее. В результате восстанавливается исходная последовательность символов *яченьлюблюмороженое*.

Описанный пример демонстрирует случай с двумя строками, который является наиболее простым. В общем случае количество строк задаётся ключом k и может принимать любое значение от 2 до длины текста. С увеличением числа строк усложняется как процесс шифрования, так и процесс анализа зашифрованного текста, однако даже при больших значениях ключа шифр остаётся нестойким к современному криптоанализу.

Схема алгоритма шифрования «Штакетник» представлена на рисунке 4.1.



Рисунок 4.1 – Схема алгоритма шифрования «Штакетник»

4.2.2 Дешифрование

Дешифрование выполняется в обратном порядке [10], [12]:

а) по длине текста шифра и ключу k определяется, сколько символов окажется на каждой строке (для этого моделируется зигзагообразный обход);

б) зашифрованный текст разделяется на части соответствующей длины и распределяется по строкам;

в) восстанавливается исходный порядок символов: по тому же зигзагообразному шаблону символы выбираются со строк по очереди;

Схема алгоритма дешифрования зашифрованного текста «Штакетником» представлена на рисунке 4.2.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		40

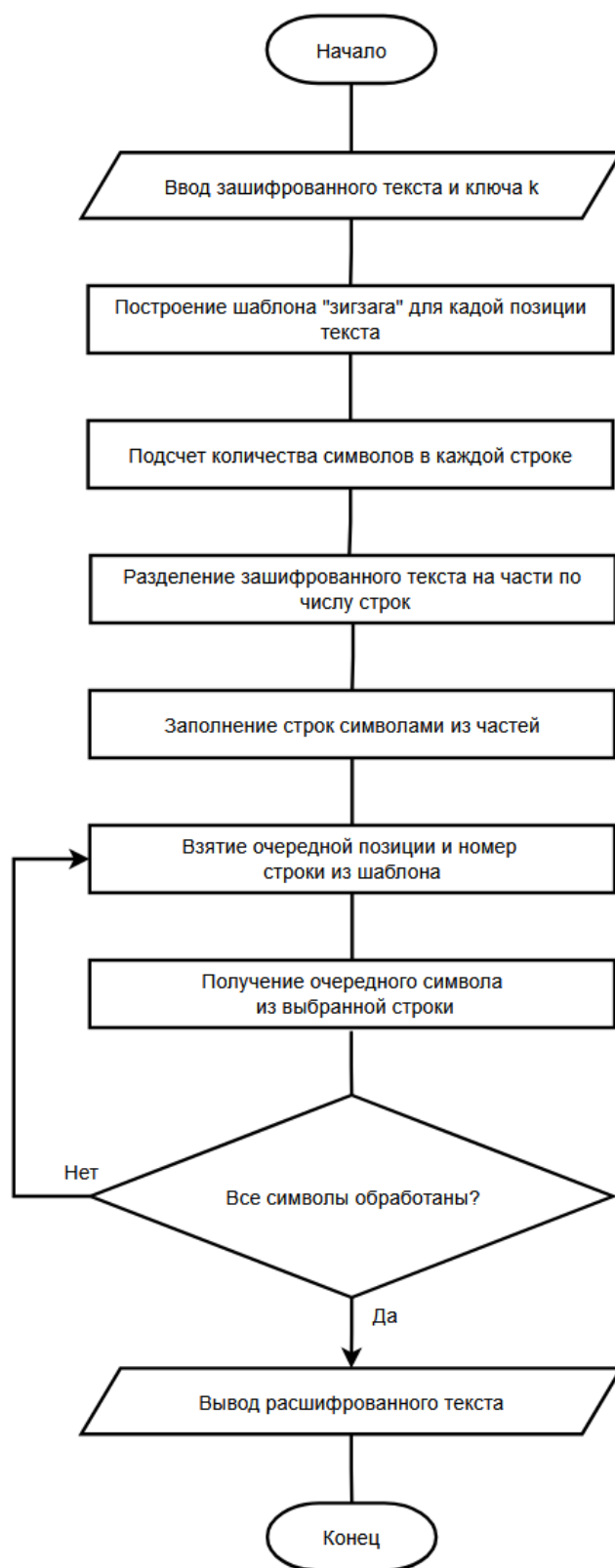


Рисунок 4.2 – Схема алгоритма дешифрования шифра «Штакетник»

4.3 Реализация шифра «Штакетник» на языке Python

В соответствии с заданием реализация шифра выполнена на чистом языке Python без использования библиотечных функций, связанных с шифрованием.

Структурно программа состоит из трёх основных компонентов: функции `encrypt`, выполняющей шифрование исходного текста, функции `decrypt`, выполняющей обратное преобразование, и главного блока, реализующего пользовательский интерфейс. Программа поддерживает работу с произвольными значениями ключа k от 2 и выше, корректно обрабатывает тексты на различных языках, включая русский и английский, а также допускает ввод символов пробелов, цифр и знаков препинания.

4.3.1 Описание работы функции шифрования

Функция `encrypt` принимает на вход исходный текст и ключ k . Если ключ меньше двух, шифрование теряет смысл и текст возвращается без изменений.

Алгоритм работает следующим образом. Создаётся список из k пустых списков. Заводятся две переменные: *rail* – текущий номер, и *direction* – направление движения по строкам (значение $+1$ означает движение вниз, -1 – вверх).

В цикле по каждому символу исходного текста символ добавляется в текущую строку. Затем номер строки увеличивается на значение направления. Если после этого номер строки достиг крайнего значения (нулевая строка или последняя $k - 1$), направление меняется на противоположное. Так и реализуется зигзагообразный обход.

После завершения обработки всех символов строки соединяются последовательно в одну строку – это и есть зашифрованный текст.

4.3.2 Описание работы функции дешифрования

Функция `decrypt` восстанавливает исходный текст по зашифрованному и ключу. Основная сложность дешифрования в том, что нужно знать, сколько символов попало на каждую строку, прежде чем приступить к восстановлению.

Сначала строится «шаблон» – массив, в котором для каждой позиции исходного текста указан номер строки, на котором этот символ оказался бы при шифровании. Шаблон строится по тому же зигзагообразному алгоритму, что и шифрование.

По шаблону подсчитывается количество символов на каждой строке. Затем текст шифра разделяется на части соответствующей длины – каждая часть восстанавливает содержимое одной строки.

В последней итерации производится восстановление исходного текста: для каждой позиции по шаблону определяется номер строки, и с этой строки берётся очередной символ. Полученный результат – расшифрованный текст.

4.4 Результаты работы программы

Для проверки корректности реализации было проведено тестирование программы на нескольких наборах данных. Результаты тестирования приведены на рисунках 4.2, 4.3 и 4.4.

```
In [1]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPractice/_Материалы д
Методические указания УТ ЗФО 2026/3/untitled0.py' --wdir
Шифр "Штакетник"
1 - шифрование
2 - дешифрование
3 - загрузить текст из файла и зашифровать
Выберите действие: 1
Введите текст: ЯЛЮБЛИКОМОРОЖЕНОЕ
Введите ключ (число рельсов): 2
Зашифрованный текст: ЯЮЛМРЖНЕЛБЮООЕО

In [2]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPractice/_Материалы д
Методические указания УТ ЗФО 2026/3/untitled0.py' --wdir
Шифр "Штакетник"
1 - шифрование
2 - дешифрование
3 - загрузить текст из файла и зашифровать
Выберите действие: 2
Введите зашифрованный текст: ЯЮЛМРЖНЕЛБЮООЕО
Введите ключ (число рельсов): 2
Расшифрованный текст: ЯЛЮБЛИКОМОРОЖЕНОЕ
```

Рисунок 4.2 – Запуск программы с тестовым набором 1

```

In [4]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPrac
Методические указания УТ ЗФО 2026/3/untitled0.py' --wdir
Шифр "Штакетник"
1 - шифрование
2 - дешифрование
3 - загрузить текст из файла и зашифровать
Выберите действие: 1
Введите текст: КРИПТОГРАФИЯЭТОИНТЕРЕСНО
Введите ключ (число рельсов): 2
Зашифрованный текст: КИТГАИЭОНЕЕНРПОРФЯТИТРСО

In [5]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPrac
Методические указания УТ ЗФО 2026/3/untitled0.py' --wdir
Шифр "Штакетник"
1 - шифрование
2 - дешифрование
3 - загрузить текст из файла и зашифровать
Выберите действие: 2
Введите зашифрованный текст: КИТГАИЭОНЕЕНРПОРФЯТИТРСО
Введите ключ (число рельсов): 2
Расшифрованный текст: КРИПТОГРАФИЯЭТОИНТЕРЕСНО

```

Рисунок 4.2 – Запуск программы с тестовым набором 2

```

In [6]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPrac
Методические указания УТ ЗФО 2026/3/untitled0.py' --wdir
Шифр "Штакетник"
1 - шифрование
2 - дешифрование
3 - загрузить текст из файла и зашифровать
Выберите действие: 1
Введите текст: HELLOWORLD
Введите ключ (число рельсов): 3
Зашифрованный текст: HOLELWRDLO

In [7]: %runfile 'C:/Users/rameshkamir/Study/vyatsu_works/02.06.2026/studyPrac
Методические указания УТ ЗФО 2026/3/untitled0.py' --wdir
Шифр "Штакетник"
1 - шифрование
2 - дешифрование
3 - загрузить текст из файла и зашифровать
Выберите действие: 2
Введите зашифрованный текст: HOLELWRDLO
Введите ключ (число рельсов): 3
Расшифрованный текст: HELLOWORLD

```

Рисунок 4.2 – Запуск программы с тестовым набором 3

Тестирование показало, что разработанная программа корректно выполняет шифрование и дешифрование текстов на различных языках (английский, русский), включая тексты с пробелами и специальными символами. Алгоритм работает с любыми допустимыми значениями ключа – от 2 до длины текста. Расшифрованный текст всегда совпадает с исходным, что подтверждает корректность реализации обеих операций.

4.5 Особенности алгоритма

Шифр «Штакетник» в современной криптографии считается крайне нестойким по нескольким причинам:

- малое число возможных ключей. Ключом является число от 2 до длины текста, что даёт небольшое количество вариантов перебора (криптоаналитик может просто перебрать все возможные значения ключа за крайне малое время);
- сохранение частотных характеристик текста. Шифр является шифром перестановки и не изменяет состав символов исходного текста. Каждая буква остаётся той же, изменяется лишь её позиция. Это позволяет применять методы частотного криптоанализа;
- наличие визуальных и статистических закономерностей в распределении символов по строкам, которые могут быть выявлены при анализе.

В настоящее время шифр «Штакетник» применяется главным образом в образовательных целях: для иллюстрации принципов классической криптографии, демонстрации идеи шифров перестановки, обучения программированию алгоритмов работы со строками. Также шифр иногда используется в развлекательных целях – в играх, головоломках, квестах.

4.6 Вывод по разделу 4

В ходе работы был изучен и реализован один из классических криптографических алгоритмов – шифр «Штакетник», относящийся к группе шифров перестановки. Программа разработана на чистом языке Python без использования специализированных криптографических библиотек, что соответствовало требованиям задания.

В процессе выполнения работы были получены следующие результаты:

- изучена история возникновения и принцип работы шифра «Штакетник», рассмотрены его место в классификации классических шифров и область применения;

– разработан и реализован программный код, выполняющий шифрование и дешифрование текста по заданному ключу;

– реализован пользовательский интерфейс, позволяющий вводить текст вручную или загружать его из файла, а также выбирать операцию шифрования или дешифрования;

– проведено тестирование программы на трех наборах данных, во всех случаях получены корректные результаты.

Изучение шифра «Штакетник» позволило освоить основные понятия классической криптографии: симметричное шифрование, ключ, шифры перестановки, криптостойкость. Также были закреплены практические навыки работы с базовыми возможностями языка Python: операции со строками и списками, циклы и условные конструкции, функции с возвратом значений, обработка пользовательского ввода и работа с файлами.

Несмотря на то, что шифр «Штакетник» не пригоден для практической защиты информации в современных условиях, его реализация имеет важное образовательное значение. Полученные знания о принципах построения шифров перестановки, понимание их преимуществ и ограничений, а также практические навыки разработки криптографического программного обеспечения служат основой для дальнейшего изучения более сложных и стойких криптографических алгоритмов.

Заключение

В ходе выполнения учебной практики были изучены базовые возможности языка программирования Python и реализованы на его основе алгоритмы кластеризации, классификации и криптографической защиты информации. Поставленные цели и задачи практики выполнены в полном объёме.

В первом разделе работы был выполнен обзор технической литературы по тематике практики, рассмотрены варианты установки интерпретатора Python, проанализированы основные среды разработки и редакторы кода для языка. Изучены базовые конструкции языка Python: типы данных и операции, условные конструкции и циклы, механизм обработки исключений, средства создания собственных функций и импорта модулей. Освоение этих конструкций обеспечило теоретическую и практическую основу для дальнейшей реализации алгоритмов.

Во втором разделе изучены теоретические основы алгоритма кластеризации k -средних и программная реализация на языке Python с использованием библиотеки NumPy. Алгоритм применён к практической задаче разделения студентов на две учебные группы по их оценкам по физике и математике. Полученные результаты соответствуют ожидаемому разделению на группы с более высоким и более низким уровнем подготовки, что подтверждает корректность реализации.

В третьем разделе изучены теоретические основы алгоритма классификации k ближайших соседей и реализована функция классификации с применением Z -нормализации признаков и евклидова расстояния. Алгоритм применён к задаче определения вида обезьян по их физическим параметрам – росту и весу. Тестирование показало, что разработанная программа корректно классифицирует особей всех четырёх рассматриваемых видов.

В четвёртом разделе изучены принципы работы классических шифров перестановки на примере шифра «Штакетник», рассмотрена история его

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		47

возникновения и место в классификации криптографических алгоритмов. Разработана программа шифрования и дешифрования текста на чистом языке Python без использования специализированных криптографических библиотек. Тестирование программы на различных наборах данных, включая тексты на русском и английском языках с разными значениями ключа, во всех случаях показало корректность шифрования и обратимое восстановление исходного текста.

Практическим результатом работы стал разработанный набор программ на языке Python, реализующих изученные алгоритмы. Полученные программы могут быть использованы в учебных целях для демонстрации принципов работы соответствующих алгоритмов.

В ходе работы были закреплены практические навыки программирования на языке Python, работы с библиотекой NumPy для научных вычислений, реализации алгоритмов с использованием базовых конструкций языка, тестирования и отладки программ. Полученные знания и навыки служат необходимой основой для дальнейшего изучения более сложных алгоритмов машинного обучения, современных методов криптографической защиты информации и разработки прикладного программного обеспечения в области информационных технологий в системах управления.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		48

Приложение А

(обязательное)

Основные типы данных в Python

Таблица А.1 – Основные типы данных Python

Название типа	Описание	Размер
int	Тип для обозначения целых чисел	28 байт, растет с увеличением значения
float	Тип для обозначения чисел с плавающей точкой	24 байта
complex	Комплексное число	32 байта
bool	Логический тип данных	28 байт
str	Текстовый тип данных	49 байт и более
list	Упорядоченная коллекция элементов произвольных типов	56 байт – пустой + 8 байт на каждый элемент
tuple	Упорядоченная неизменяемая коллекция элементов произвольных типов	40 байт – пустой + 8 байт на каждый элемент
range	Неизменяемая последовательность целых чисел в заданном диапазоне	Зависит от заданного диапазона
bytes	Неизменяемая последовательность байтов	33 байта – пустой + 1 байт на каждый символ
dict	Коллекция пар «ключ – значение»	64 байта (пустой), растет блоками
NoneType	Тип, обозначающий отсутствие значения	16 байт

Приложение Б

(обязательное)

Листинг программного кода реализации алгоритма k -средних

Листинг Б.1 – Исходный код реализации алгоритма k средних

```
import numpy as np

def dist(A, B):
    return np.sqrt(np.sum((A - B) ** 2 ))

def class_of_each_point(X, centers):
    m = len(X)
    k = len(centers)

    distances = np.zeros((m, k))
    for i in range(m):
        for j in range(k):
            distances[i, j] = dist(centers[j], X[i])

    return np.argmin(distances, axis=1)

def kmeans(k, X):
    m, n = X.shape
    curr_iteration = prev_iteration = np.zeros(m)

    x_min = np.min(X, axis=0)
    x_max = np.max(X, axis=0)
    centers = np.random.random((k, n)) * (x_max - x_min) + x_min

    curr_iteration = class_of_each_point(X, centers)

    while np.any(curr_iteration != prev_iteration):
        prev_iteration = curr_iteration

        for i in range(k):
            sub_X = X[curr_iteration == i, :]
            if len(sub_X) > 0:
                centers[i, :] = np.mean(sub_X, axis=0)

        curr_iteration = class_of_each_point(X, centers)

    return centers
```

Приложение В

(обязательное)

Таблица с данными для решения задачи классификации

Таблица В.1 – Экспериментальные данные профессора Икота

Номер	Рост	Вес	Вид
1	33	21	Лемур
2	41	13	Лемур
3	18	22	Лемур
4	38	34	Лемур
5	62	118	Шимпанзе
6	59	137	Шимпанзе
7	95	131	Шимпанзе
8	83	110	Шимпанзе
9	185	155	Горилла
10	193	129	Горилла
11	164	135	Горилла
12	205	131	Горилла
13	145	55	Орангутан
14	168	35	Орангутан
15	135	47	Орангутан
16	138	66	Орангутан

Приложение Г

(обязательное)

Листинги программного кода алгоритма k ближайших соседей

Листинг Г.1 – Исходный код файла run.py

```
import numpy as np
from knn import k_nearest

X = np.array([
    [33, 21, 1],
    [41, 13, 1],
    [18, 22, 1],
    [38, 34, 1],
    [62, 118, 2],
    [59, 137, 2],
    [95, 131, 2],
    [83, 110, 2],
    [185, 155, 3],
    [193, 129, 3],
    [164, 135, 3],
    [205, 131, 3],
    [145, 55, 4],
    [168, 35, 4],
    [135, 47, 4],
    [138, 66, 4],
])

height = int(input('Введите рост особи: '))
weight = int(input('Введите вес особи: '))
obj = np.array([height, weight])

k = 3
object_class = k_nearest(X, k, obj)

monkeys = {1: 'lemur', 2: 'schimpanze', 3: 'gorilla', 4: 'orangutan'}
print(monkeys[object_class])
```

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		52

Листинг Г.2 – Исходный код файла k_nearest.py

```
import numpy as np
import math

def k_nearest(X, k, obj):
    sub_X = X[:, 0:-1].astype(float)

    mean = np.mean(sub_X, axis=0)
    std = np.std(sub_X, axis=0)
    sub_X = (sub_X - mean) / std

    obj_norm = (obj - mean) / std

    distances = [dist(obj_norm, sub_X[i]) for i in range(len(sub_X))]
    sorted_indices = np.argsort(distances)
    nearest_classes = X[sorted_indices[:k], -1]
    unique, counts = np.unique(nearest_classes, return_counts=True)
    object_class = unique[np.argmax(counts)]

    return int(object_class)

def dist(p1, p2):
    return math.sqrt(sum((p1 - p2) ** 2))
```

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		53

Приложение Д

(обязательное)

Листинг программного кода реализации алгоритма шифрования «Штакетник»

Листинг Д.1 – Исходный код файла main.py

```
def encrypt(text, key):
    if key < 2:
        return text

    rails = [[] for _ in range(key)]

    rail = 0
    direction = 1

    for char in text:
        rails[rail].append(char)
        rail += direction

        if rail == key - 1 or rail == 0:
            direction = -direction

    result = ''.join(''.join(r) for r in rails)
    return result


def decrypt(ciphertext, key):
    if key < 2:
        return ciphertext

    length = len(ciphertext)

    pattern = [0] * length
    rail = 0
    direction = 1
    for i in range(length):
        pattern[i] = rail
        rail += direction
        if rail == key - 1 or rail == 0:
            direction = -direction

    rail_counts = [0] * key
    for r in pattern:
        rail_counts[r] += 1

    rails = []
    pos = 0
    for r in range(key):
        rails.append(list(ciphertext[pos:pos + rail_counts[r]]))
        pos += rail_counts[r]
```

```

rail_indices = [0] * key
result = []
for i in range(length):
    r = pattern[i]
    result.append(rails[r][rail_indices[r]])
    rail_indices[r] += 1

return ''.join(result)

if __name__ == '__main__':
    print('Шифр «Штакетник»')
    print('1 - шифрование')
    print('2 - дешифрование')
    print('3 - загрузить текст из файла и зашифровать')

    choice = input('Выберите действие: ')

    if choice == '1':
        text = input('Введите текст: ')
        key = int(input('Введите ключ (число строк): '))
        result = encrypt(text, key)
        print(f'Зашифрованный текст: {result}')
    elif choice == '2':
        text = input('Введите зашифрованный текст: ')
        key = int(input('Введите ключ (число строк): '))
        result = decrypt(text, key)
        print(f'Расшифрованный текст: {result}')
    elif choice == '3':
        filename = input('Введите имя файла: ')
        with open(filename, 'r', encoding='utf-8') as f:
            text = f.read()
        key = int(input('Введите ключ (число строк): '))
        result = encrypt(text, key)
        print(f'Зашифрованный текст: {result}')
    else:
        print('Неизвестная команда')

```

Приложение Е

(справочное)

Перечень определений и терминов

динамическая типизация – особенность языка программирования, при которой тип переменной определяется автоматически на основе присвоенного значения и может изменяться в процессе выполнения программы;

кортеж – упорядоченная неизменяемая коллекция элементов произвольных типов в языке Python;

исключение – ошибка или иная исключительная ситуация, возникающая во время выполнения программы и требующая специальной обработки;

рекурсия – способ организации функции, при котором она вызывает саму себя для решения задачи через её сведение к более простым случаям;

кластер – группа объектов, обладающих внутренней однородностью по признакам и внешней изолированностью от других групп;

евклидово расстояние – мера расстояния между двумя точками в N-мерном пространстве, вычисляемая как корень из суммы квадратов разностей их координат;

Z-нормализация – способ предварительной обработки данных, при котором значения каждого признака приводятся к нулевому среднему и единичному среднеквадратическому отклонению;

шифр перестановки – класс криптографических алгоритмов, в которых символы исходного текста сохраняются, но изменяется их порядок согласно заданному правилу;

ключ шифрования – параметр криптографического алгоритма, определяющий конкретное преобразование при шифровании и дешифровании сообщения.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		56

Приложение Ж

(справочное)

Перечень сокращений

В настоящей работе используются следующие сокращения:

ИИ – искусственный интеллект;

СУБД – система управления базами данных;

IDE – Integrated Development Environment, интегрированная среда разработки;

ISBN – International Standard Book Number, международный стандартный книжный номер;

PEP – Python Enhancement Proposal, предложение по развитию Python;

URL – Uniform Resource Locator, единый указатель ресурса.

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		57

Приложение И

(справочное)

Библиографический список

1. The Python Tutorial [Электронный ресурс] // Python Software Foundation, 2026. URL: <https://docs.python.org/3/tutorial/> (дата обращения: 21.05.2026).

2. Лутц М. Изучаем Python. Том 1 / М. Лутц ; пер. с англ. – 5-е изд. – Москва ; Санкт-Петербург : Диалектика, 2019. – 832 с. – ISBN 978-5-907144-52-1.

3. Python Tutorials [Электронный ресурс] // Real Python, 2026. URL: <https://realpython.com/> (дата обращения: 21.05.2026).

4. Index of Python Enhancement Proposals [Электронный ресурс] // Python Software Foundation, 2026. URL: <https://peps.python.org/> (дата обращения: 22.05.2026).

5. NumPy Documentation [Электронный ресурс] // NumPy Developers, 2026. URL: <https://numpy.org/doc/stable/> (дата обращения: 22.05.2026).

6. Matplotlib Documentation [Электронный ресурс] // The Matplotlib Development Team, 2026. URL: <https://matplotlib.org/stable/index.html> (дата обращения: 22.05.2026).

7. Маккинни У. Python и анализ данных : первичная обработка данных с применением pandas, NumPy и Jupyter / У. Маккинни ; пер. с англ. А. А. Слинкина. – 3-е изд. – Москва : ДМК Пресс, 2023. – 536 с. – ISBN 978-5-93700-174-0.

8. scikit-learn: Machine Learning in Python [Электронный ресурс] // scikit-learn developers, 2026. URL: <https://scikit-learn.org/stable/> (дата обращения: 25.05.2026).

9. Воронцов К. В. Машинное обучение (курс лекций) [Электронный ресурс] // MachineLearning.ru, 2024. URL: [http://www.machinelearning.ru/wiki/index.php?title=Машинное_обучение_\(курс_лекций,_К.В.Воронцов\)](http://www.machinelearning.ru/wiki/index.php?title=Машинное_обучение_(курс_лекций,_К.В.Воронцов)) (дата обращения: 25.05.2026).

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		58

10. Основы криптоанализа шифра простой перестановки [Электронный ресурс] // Habr, 2025. URL: <https://habr.com/ru/articles/881580/> (дата обращения: 25.05.2026).

11. Шифрование сообщений в Python. От простого к сложному. Шифр Цезаря [Электронный ресурс] // Habr, 2021. URL: <https://habr.com/ru/articles/552212/> (дата обращения: 25.05.2026).

12. Rail fence cipher [Электронный ресурс] // Wikipedia, 2024. URL: https://en.wikipedia.org/wiki/Rail_fence_cipher (дата обращения: 25.05.2026).

					ТПЖА.270304.259 ПЗ	Лист
Изм	Лист	№ докум.	Подпись	Дата		59